

OOP Finals 2025

პალინდრომი (სტრინგები) - 30 ქულა

დააიმიპლემენტირეთ `findLongestPalindrome` მეთოდი, რომელსაც გადაეცემა სტრინგების სია და აბრუნებს ყველაზე გრძელ პალინდრომს, ან ცარიელ სტრინგს თუ პალინდრომი საერთოდ არ მოიძებნა.

```
public String findLongestPalindrome(List<String> words) {  
    return "";  
}
```

- პალინდრომია სტრინგი, რომელიც წინიდანაც და ბოლოდანაც ერთნაირად იკითხება.
- სტრინგები შეიცავს მხოლოდ ლათინური ანბანის დიდი და პატარა სიმბოლოებს.
- შემოწმებისას არ უნდა გაითვალისწინოთ სიმბოლოების რეგისტრი, მაგალითად "AbacABA" პალინდრომია.
- მაგალითად: ("madam", "racecar", "apple", "baNaNa", "Detartrated", "Level") --> "Detartrated"

ამოცანა ამოხსენით მხოლოდ სტრინგებით და ერთადერთი return statement-ით. არ შექმნათ დამატებითი ცვლადები/მეთოდები/კლასები...

მატრიცების გამრავლება (ნაკადები) - 50 ქულა

დანერეთ `parallelMatrixMultiply` მეთოდი რომელსაც გადაეცემა 2 მატრიცი ($A[n][n]$ - $m \times n$ და $B[n][n]$ - $n \times p$) და აბრუნებს მატრიცების ნამრავლს ($m \times p$ განზომილების).

ამოცანა უნდა გადაიჭრას $m * n * p$ რაოდენობის ნაკადით (**მაქსიმალური პარალელიზაციით, იტერაციული ამოხსნა არ ჩაითვლება**), სადაც თითოეული ნაკადი ითვლის რა რიცხვი უნდა დაემატოს საშუალო C მატრიცის კონკრეტულ უჯრაზე. მაგალითად $A[0][0]$ და $B[0][0]$ ის ნამრავლი ემატება $C[0][0]$ -ზე, თუმცა $C[0][0]$ -ზე ასევე დაემატება $A[0][1] * B[1][0]$ რომელიც სხვა ნაკადმა უნდა შეასრულოს.

გამრავლების ალგორითმი:

მატრიცის ელემენტი $C[i][j]$ ითვლება ასე:

$$C[i][j] = A[i][0] * B[0][j] + A[i][1] * B[1][j] + \dots + A[i][n-1] * B[n-1][j]$$

ანუ, პირველი მატრიცის სტრიქონებს ვამრავლებთ მეორე მატრიცის სვეტებზე და შედეგებს ვკრეფთ C მატრიცის შესაბამის უჯრაში.

მაგალითად:

A = [[1, 2, 3],
[4, 5, 6]]

B = [[7, 8],
[9, 10],
[11, 12]]

C = [[58, 64],
[139, 154]]

$C[0][0] = 1 \times 7 + 2 \times 9 + 3 \times 11 = 58$

$C[0][1] = 1 \times 8 + 2 \times 10 + 3 \times 12 = 64$

$C[1][0] = 4 \times 7 + 5 \times 9 + 6 \times 11 = 139$

$C[1][1] = 4 \times 8 + 5 \times 10 + 6 \times 12 = 154$

ამ მაგალითისთვის გვექნება 12 ნაკადი. თითოეული შესაბამის უჯრაზე დაამატებს თითო ნამრავლს.

ქვემოთ მოცემული კოდი სწორად ითვლის მატრიცების ნამრავლს, მაგრამ მუშაობს მხოლოდ ერთი ნაკადით, ამიტომ არ აკმაყოფილებს ამოცანის მოთხოვნებს:

```
for (int i = 0; i < m; i++) {  
    for (int j = 0; j < p; j++) {  
        for (int k = 0; k < n; k++) {  
            C[i][j] += A[i][k] * B[k][j];  
        }  
    }  
}
```

OOP - 50 ქულა

მოცემულია ცუდი კოდი, რომელიც ითვლის გეომეტრიული ფიგურების მოცულობებს.

თქვენი მიზანია მაქსიმალურად გამოასწოროთ ეს კოდი OOP პრინციპების დაცვით.

```

public class ShapeCalculator {

    public void calculateAndPrintVolume(String type, double a, double b) {
        if (type.equals("sphere")) {
            double radius = a;
            double volume = (4.0 / 3.0) * Math.PI * Math.pow(radius, 3);
            System.out.println("Shape: Sphere, Volume: " + volume);
        } else if (type.equals("cube")) {
            double side = a;
            double volume = Math.pow(side, 3);
            System.out.println("Shape: Cube, Volume: " + volume);
        } else if (type.equals("cylinder")) {
            double radius = a;
            double height = b;
            double volume = Math.PI * Math.pow(radius, 2) * height;
            System.out.println("Shape: Cylinder, Volume: " + volume);
        } else {
            System.out.println("Unknown shape");
        }
    }

    public static void main(String[] args) {
        ShapeCalculator sc = new ShapeCalculator();

        sc.calculateAndPrintVolume("sphere", 5, 0);
        sc.calculateAndPrintVolume("cube", 3, 0);
        sc.calculateAndPrintVolume("cylinder", 2, 10);
    }
}

```

არ გამოიყენოთ **branch statements** როგორიცაა **if/else, switch**. მათ ნაცვლად შექმენით კლასები(შეგიძლიათ 1 ფაილში ჩამოწეროთ ყველა კლასი). ასევე საბოლოო **Main** მეთოდში შესაძლებელი უნდა იყოს ფიგურების სიის მარტივად დამუშავება და **for** ციკლით თითოეული ფიგურის მონაცემების დაბეჭდვა.

ონლაინ აუქციონი (ვებ არქიტექტურა) - 70 ქულა

თქვენი ამოცანაა მოიფიქროთ ინტერნეტ აუქციონის გამარტივებული ვერსიის ვებ არქიტექტურა. ქვემოთ მოცემულია ძირითადი მოთხოვნები

მომხმარებლების ტიპები

- **ადმინისტრატორი** - ადმინისტრატორს უნდა შეეძლოს აუქციონების მენეჯმენტი (დამატება, რედაქტირება, გაუქმება)
ადმინისტრატორის ანგარიშებს ქმნის მხოლოდ სისტემის ადმინისტრაცია (საჯარო რეგისტრაცია ადმინისტრაციის არ შეიძლება).
- **მომხმარებელი** - მომხმარებელს უნდა შეეძლოს რეგისტრაცია, სისტემაში შესვლა, აუქციონების ძებნა და მონაწილეობა (რაც გულისხმობს ბიდების, ანუ ფასის შედავაზებების გაკეთებას)

აუქციონების მართვა

- აუქციონის შესახებ უნდა ინახებოდეს შემდეგი მინიმალური ინფორმაცია:
 - სათაური
 - აღწერა
 - სურათები
 - სანყისი ფასი
 - დანყების და დამთავრების დროები
- მომხმარებლებს უნდა შეეძლოთ:
 - აუქციონების ძებნა(დროით, სტატუსით, სათაურით...)
 - თითოეული აუქციონის დეტალების ნახვა
 - ბიდების გაკეთება
- სისტემა უნდა ინახავდეს
 - ბიდების ისტორიას
 - მიმდინარე ყველაზე მაღალი ბიდს და მის ავტორს
- მოკლედ ახსენით, როგორ უზრუნველყოფდით სისტემაში კონკურენტულობას? მაგალითად რა ზომებს მიიღებდით იმისთვის, რომ ორმა მომხმარებელმა ერთდროულად ვერ შეძლოს გამარჯვებული ბიდის დადება?

ბიდების მართვა

- ბიდის დადება შესაძლებელია მხოლოდ იმ აუქციონზე, რომელიც აქტიურია (არ დასრულებულა)
- მიღებული ბიდი უნდა იყოს მიმდინარე ფასზე მაღალი
- აუქციონის დასრულების შემდეგ, გამარჯვებული ყველაზე მაღალი ბიდის ავტორი.
- სისტემამ გამარჯვებულ მომხმარებელს უნდა შეატყობინოს აუქციონის მოგების შესახებ

გაითვალისწინეთ რომ ვებსაიტი დაცული უნდა იყოს, ანუ აუქციონების მართვა მარტო ადმინისტრატორს უნდა შეეძლოს, ხოლო აუქციონებში მონაწილეობის მიღება - მხოლოდ მომხმარებლებს

ნამუშევრის ორგანიზება:

- **db.txt** - ფაილში ჩანერეთ თუ რა ცხრილები დაგჭირდებათ ბაზაში და თითოეულ ცხრილს რა სქემა ექნება. გამართული SQL-ით აღწერა არ მოგეთხოვებათ, საკმარისია თითოეული ცხრილისთვის ჩამოთვალეთ ველების სახელები, ტიპები, **primary** და **foreign** გასაღებები.
- **urls.txt** - ფაილში ჩანერეთ თუ რა URL-ებს მოემსახურება თქვენი იმპლემენტაცია და ყოველ URL-ს მიუთითეთ თუ რომელი სერველეტი მოემსახურება მას.
- თითოეულ სერველტს მოკლედ მიუთითეთ თუ რა ფუნქციონალს უკეთებს იმპლემენტაციას და როგორ. როგორში იგულისხმება ბაზასთან კავშირს როგორ ამყარებს, სესიაში ინახავს თუ არა რამეს, თავს როგორ იცავს ანუ მომხმარებლების უფლებებს როგორ ამოწმებს. სერველტების აღწერები შეგიძლიათ იგივე **urls.txt** ფაილში გააკეთოთ ან შექმნათ ცალკე **java** ფაილები. არ მოგეთხოვებათ მათი სრული იმპლემენტაცია, საკმარისია მიუთითოთ სხვადასხვა HTTP ტიპის მოთხოვნებზე როგორ იქცევა.
- თუ თქვენს იმპლემენტაციას ჭირდება **Servlet context**-ის ან სესიების გამოყენება, თითოეულისთვის აღწერეთ რას ინახავთ მასში და როგორ იყენებთ.

შეფასებისას ასევე ყურადღება მიექცევა ბაზასთან კავშირს რა ინტერფეისებით და მოდელის კლასებით ააწყობთ. საკმარისია ამ კლასების ინტერფეისების ახსნა, დეტალური იმპლემენტაცია არ არის საჭირო.